第1章作业下周交，习题集1.6 1.7 1.8 1.10 1.17 1.20

↓

各班学习委员

↓ ↓ ↓

助教：

第1章回顾

数据结构课程—— 数据结构 + 算法 = 程序，涉及数学、计算机硬件和软件。

数据结构定义——指互相有关联的数据元素的集合，可用data_Structure=(D,R)表示。

数据结构内容——数据的逻辑结构、存储结构和基本运算(计算机处理非数值对象)

数据结构学习工具——抽象数据类型和伪码（类C）

算法效率指标——时间效率和空间效率

目　　录

第1章　绪论
第2章　线性表
第3章　栈和队列
第4章　串
第5章　数组和广义表
第6章　树和二叉树
第7章　图
第9章　查找
第10章　排序

数据结构课程的起点：

- 数据结构
  - 逻辑结构
    - 线性结构（线性表、栈 、队、串、数组）
    - 非线性结构
      - 树结构
      - 图结构
  - 物理（存储）结构
    - 顺序结构
    - 链式结构
    - 索引结构
    - 散列结构
  - 数据运算
    - 插入运算
    - 删除运算
    - 修改运算
    - 查找运算
    - 排序运算

什么是线性结构?

线性结构的定义：

若结构是非空有限集，则有且仅有一个开始结点和一个终端结点，并且所有结点都最多只有一个直接前趋和一个直接后继。 可表示为： （a_1， a_2 ，……, a_n）

特点① 只有一个首结点和尾结点；

特点② 除首尾结点外，其他结点只有一个直接前驱和一个直接后继。

简言之，线性结构反映结点间的逻辑关系是一对一 (1:1) 的。

线性结构包括：线性表、堆栈、队列、字符串、数组等，其中最典型、最常用的是------ 线性表

第2章　线性表

2.1　线性表的逻辑结构

2.2　线性表的顺序表示和实现

2.3　线性表的链式表示和实现

2.4　应用举例

2.1 线性表的逻辑结构

线性表的定义：用数据元素的有限序列表示

$$(a_1, a_2, \ldots a_{i-1}, a_i, a_{i+1}, \ldots, a_n)$$

数据元素

线性起点

a_i的直接前趋

a_i的直接后继

线性终点

下标，是元素的序号，表示元素在表中的位置

n=0时称为 空表

n为元素总个数，即表长。$n \geq 0$

例1　分析26 个英文字母组成的英文表是什么结构。

分析：　数据元素都是同类型（字母），　元素间关系是线性的。

（A，B，C，D, ……，Z)

例2　分析学生情况登记表是什么结构。

学号	姓名	性别	年龄	班级
U200713569	李春元	男	19	通信工程200701班
U200713611	李煜墉	男	19	通信工程200701班
U200713537	张祺轩	男	19	通信工程200702班
U200713564	李姜辉	男	19	通信工程200703班
U200713582	高　岩	男	19	通信工程200704班
:	:	:	:	:

分析：数据元素都是同类型（记录），元素间关系是线性的。

注意：同一线性表中的元素必定具有相同特性！

试判断下列叙述的正误：

“同一数据逻辑结构中的所有数据元素都具有相同的特性”是指数据元素所包含的数据项的个数都相等。

是指各元素具有相同的数据类型

2.2 线性表的顺序表示和实现

2.2.1 顺序表的表示

2.2.2 顺序表的实现

2.2.3 顺序表的运算效率分析

2.2.1 顺序表的表示

线性表的顺序表示又称为顺序存储结构或顺序映像。

顺序存储定义：**把逻辑上相邻的数据元素存储在物理上相邻的存储单元中的存储结构。**

特点：**逻辑上相邻的元素，物理上也相邻**

顺序存储方法：**用一组地址连续的存储单元依次存储线性表的元素。**

可以利用数组V[n]来实现

注意：在C语言中数组的下标是从0开始，即：V[n]的有效范围是从 V[0] ~ V[n-1]

线性表顺序存储特点：

1. 逻辑上相邻的数据元素，其物理上也相邻；

2. 若已知表中首元素在存储器中的位置，则其他元素存放位置亦可求出（利用数组V[n]的下标）。

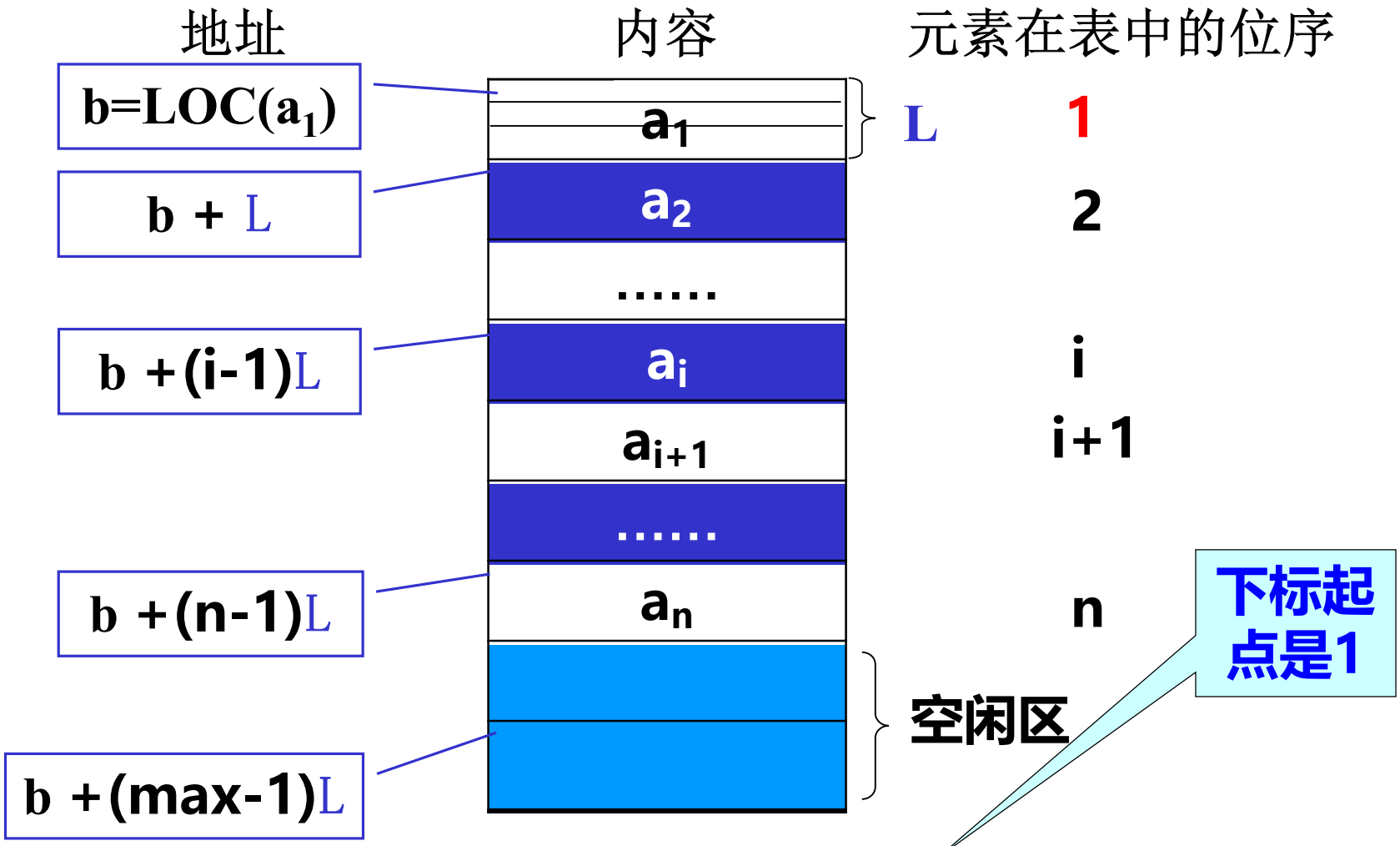
设首元素a_1的存放地址为$LOC(a_1)$（称为首地址），设每个元素占用存储空间（地址长度）为L字节，则表中任一数据元素的存放地址为：

$$LOC(a_{i+1}) = LOC(a_i) + L$$

$$LOC(a_i) = LOC(a_1) + L * (i-1)$$

对上述公式的解释如图所示

线性表的顺序存储结构示意图



$$LOC(a_i) = LOC(a_1) + L * (i-1)$$

例1　设有一维数组M，下标的范围是0到9，每个数组元素用相邻的5个字节存储。存储器按字节编址，设存储数组元素M[0]的第一个字节的地址是98，则M[3]的第一个字节的地址是多少?

答案：113

解：已知地址计算通式为：

$$LOC(a_i) = LOC(a_1) + L * (i-1)$$

但此题要注意下标起点是从0开始：

$$LOC(M_{[3]}) = 98 + 5 \times (4-1) = 113$$

或用（3-0）

例2　用数组V来存放26个英文字母组成的线性表（a，b，c，…，z），写出在顺序结构上生成和显示该表的C语言程序。

省略了include语句

```
char V[30];
void build()    //字母线性表的生成，即建表操作
{
    int i;
    V[0]='a';
    for( i=1;   i<=n-1;   i++ )
    V[i]=V[i-1]+1;
}
```

核心语句：

法1　V[i]= V[i-1]+1;

法2　V[i]='a'+i;

法3　V[i]=97+i;

```
void display( ) //字母线性表的显示，即读表操作
{
    int i;
    for( i=0;  i<=n-1;  i++ )
    printf( "%c",  v[i] );
    printf( "\n " );
}
void main(void)    //主函数，字母线性表的生成和输出
{
     n=26; // n是表长，是数据元素的个数，而不是V的实际下标
     build( );
    display( );
}
```

执行结果： a b c d e f g h i j k l m n o p q r s t u v w x y z

2.2.2 顺序表的实现（或操作）

数据结构的基本运算：

修改、插入、删除、查找、排序

1) 修改 **通过数组的下标便可访问某个特定元素并修改之。**

核心语句: **V[i]=x;**

显然，顺序表修改操作的时间效率是 **O(1)**

2)插入　　在线性表的第i个位置前插入一个元素

实现步骤：

- 将第n至第i 位的元素向后移动一个位置；
- 将要插入的元素写到第i个位置；
- 表长加1。

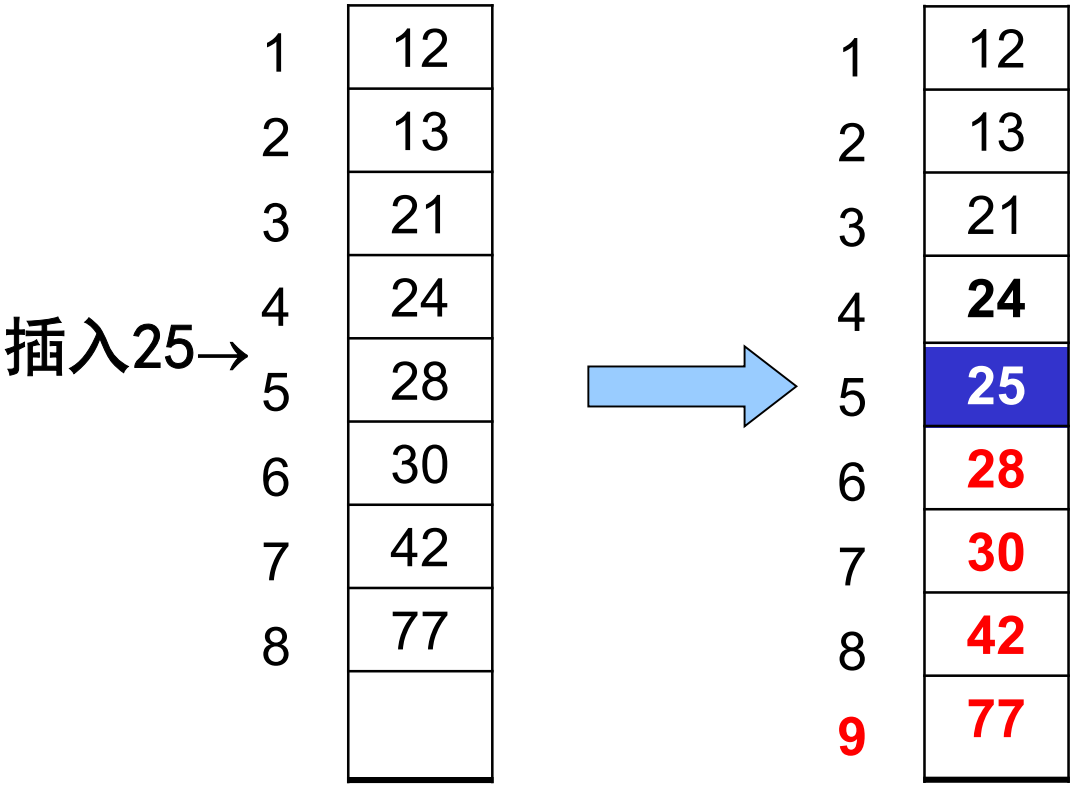
注意：事先应判断: 插入位置i 是否合法?表是否已满?

应当符合条件：　$1 \le i \le n+1$　或　$i=[1, n+1]$

核心语句：

```
for (j=n; j>=i; j--)
a[j+1]=a[ j ];          // 元素后移一个位置
a[ i ]=x;               // 插入x
n++;                    // 表长加1
```

在线性表的第i个位置前插入一个元素的示意图如下：



3)删除　删除线性表的第i个位置上的元素

实现步骤：

- 将第i+1 至第n 位的元素向前移动一个位置；
- 表长减1。

注意：事先需要判断，删除位置i 是否合法?

应当符合条件：1≤i≤n　或　i=[1, n]

核心语句：

```
for ( j=i+1; j<=n; j++ )
a[j-1]=a[j];          // 元素前移一个位置
n--;                  // 表长减1
```

删除顺序表中某个指定的元素的示意图如下：

1	12
2	13
3	21
4	24
5	25
6	28
7	30
8	42
9	77

1	12
2	13
3	21
4	24
5	28
6	30
7	42
8	77

顺序表插入和删除的完整程序请同学们自编，并务必上机验证！

2.2.3 顺序表的运算效率分析

时间效率分析:

- **算法时间主要耗费在移动元素的操作上，因此计算时间复杂度的基本操作（最深层语句频度）**

$$T(n)= O(\text{移动元素次数})$$

而移动元素的个数取决于插入或删除元素的位置。

讨论1：若在长度为 n 的线性表的第 i 位前 插入一个元素，则向后移动元素的次数f(n)为:

$$f(n) = n - i + 1$$

思考：若插入在尾结点之后，则根本无需移动（特别快）；
若插入在首结点之前，则表中元素全部要后移（特别慢）；
应当考虑在各种位置插入（共n+1种可能）的平均移动次数才合理。

推导：假定在每个元素位置上插入x的可能性都一样（即概率P相同），则应当这样来计算平均执行时间：

将所有位置的执行时间相加，然后取平均。

若在首结点前插入，需要移动的元素最多，后移次数为n；
若在a_1后面插入，要后移n-1个元素，后移次数为n-1；
……
若在a_{n-1}后面插入，后移次数为1；
若在尾结点a_n之后插入，则后移次数为0；

所有可能的元素移动次数合计： $0+1+\ldots+n = n(n+1)/2$

共有多少种插入形式?——连头带尾有n+1种!

故插入时的平均移动次数为：$n(n+1)/2 \div (n+1) = n/2 \approx O(n)$

同理可证：顺序表删除一元素的时间效率为:

T（n)=(n-1)/2 ≈O(n)

教材P25算法2.5也是对执行效率的推导：

插入效率：

$$E_{is} = \sum_{i=1}^{n+1} p_i(n-i+1) = \frac{1}{n+1}\sum_{i=1}^{n+1}(n-i+1) = \frac{n}{2}$$

删除效率：

$$E_{dl} = \sum_{i=1}^{n} q_i(n-i) = \frac{1}{n}\sum_{i=1}^{n}(n-i) = \frac{n-1}{2}$$

含义：对于顺序表，插入、删除操作平均需要移动一半元素(n / 2)

即插入、删除算法的平均时间复杂度为 O(n)

想一想：

顺序表插入、删除算法的平均空间复杂度为多少？ O(1)

因为没有占用辅助空间！

深入讨论1：

顺序表各种操作算法的“通式” 该如何书写?

———采用抽象数据类型来表示

线性表的抽象数据类型定义（见教材P19）

ADT List {

数据对象：$D=\{a_i \mid a_i \in ElemSet,\ i=1, 2, \ldots, n,\ n\geq 0\}$

数据关系：$R=\{< a_{i-1}, a_i > \mid a_{i-1}, a_i \in D,\ i=2,\ldots, n\}$

基本操作：

- 初始化、撤销、清空、判空；
- 求表长、表头、表尾、前趋、后继；
- 读元素、查找（含定位）、遍历；
- 插入、删除

} **ADT List**

线性表的基本操作如何表示？ **(见教材P19)**

- 初始化、撤销、清空、判空；
- 求表长、表头、表尾、前趋、后继；
- 读元素、查找（含定位）、遍历；
- 插入、删除

```
InitList( &L );              //建空表，初始化
DestoryList( &L );           //撤销表，释放内存
int LengthList( L );         //求表中元素个数，即表长
POSITION LocateElem (L,ElemType e，compare() ) //查找e
PriorElem( L, cur_e, &pre_e ); //求当前元素e的前驱
NextElem( L, cur_e, &next_e ); //求当前元素e的后继
ListInsertBefore(&L, i, e );      //把e插入到第i个元素之前
ListDelete( &L, i,&e );           //删除第i个元素并“看”此元素
ListTraverse( L, Visit() );       // “看”表中全部元素（遍历）
```